

Blockchain-based Provenance for Artificial Intelligence Lifecycle Traceability

Lakshitha Gunasekara*, Daswin De Silva*, Nishan Mills*, Harsha Moraliyage*, Andrew Jennings*

*Centre for Data Analytics and Cognition, La Trobe University, Australia

Abstract—The rapid adoption of Artificial Intelligence (AI) systems has increased risks associated with their development, deployment, and operation. Provenance of artefacts across the AI lifecycle is critical for addressing these risks through traceability. Although existing blockchain-based provenance systems provide immutable ledgers, they are passive due to the lack of enforcement at runtime and computationally expensive when evaluation is performed on-chain. To address these gaps, we present a novel approach based on a Merkle-style directed acyclic graph (Merkle-DAG) with off-chain constraint evaluation and on-chain enforcement. The Merkle-DAG represents process-level provenance artefacts from the full AI lifecycle, including datasets, features, models, metrics, and human oversight records such as reviews and approvals. This graph is evaluated off-chain against constraints by Open Policy Agent, followed by on-chain enforcement by a smart contract. The proposed approach is empirically demonstrated on three lifecycle stages, development, validation, and approval, of an energy consumption forecasting task in a real-world microgrid setting.

Index Terms—Blockchain, Responsible AI, MLOps, Data Provenance, Smart Contracts

I. INTRODUCTION

Artificial Intelligence (AI) systems increasingly influence decision-making in the critical infrastructure settings of energy, transport, and public safety [1]. As these AI systems increase in capability, they become more complex, opaque and less traceable or auditable [2], [3]. This increases risks, and when an unexpected incident occurs, it is highly challenging to trace data, models, metrics and human decisions used to build and deploy an AI system, including if evaluation criteria were actually satisfied, or if artefacts were modified after validation. Addressing these risks requires traceability, the ability to reconstruct the complete lineage of AI artefacts from deployment back to source data.

Provenance, the documented history of origins and transformations of an artefact [4], is fundamental to achieving traceability. In AI lifecycles, provenance must capture not only datasets and models but also feature engineering steps, evaluation metrics, and approval records [5], [6]. Blockchain technology offers a promising foundation for provenance systems due to its immutability and tamper-evident properties [7]. However, existing blockchain-based approaches exhibit two persistent limitations. The first limitation is passivity. Systems such as ProvChain [8] and SmartProvenance [9] function as tamper-evident notaries, recording lineage after the fact but offering no mechanism to prevent non-compliant artefacts from progressing through the lifecycle. A model that fails validation can still be deployed if the pipeline architecture

does not strictly enforce the constraint. The second limitation is computational cost. Approaches that attempt on-chain policy evaluation or verifiable computation incur excessive overhead, making them impractical for AI lifecycles where models are retrained frequently and evaluated against multiple criteria [7], [10].

To address these gaps, we propose an approach that combines off-chain constraint evaluation with on-chain enforcement. AI lifecycle artefacts, including datasets, features, models, and metrics, are structured as a Merkle-style Directed Acyclic Graph (Merkle-DAG). Each node is content-addressed, and child nodes cryptographically commit to their parents, ensuring that any upstream modification invalidates downstream hashes. At each lifecycle stage, a Merkle root is computed as a compact commitment to the complete provenance state. Lifecycle transitions are gated by constraint evaluation. An Open Policy Agent (OPA) evaluates the current provenance state against predefined constraints off-chain, producing a signed attestation if requirements are satisfied. A smart contract then verifies this attestation and enforces the transition on-chain, anchoring the new Merkle root only if the cryptographic proof is valid. This separation keeps constraint logic flexible and editable while ensuring that enforcement remains tamper-evident and immutable.

The contributions of this paper are, 1) A Merkle-DAG structure that captures the full AI lifecycle, from datasets, features, models, and metrics to human interactions including reviews and approvals, as a tamper-evident provenance structure, 2) A hybrid architecture combining off-chain constraint evaluation via OPA with on-chain enforcement via smart contracts, enabling active traceability, and 3) An empirical validation through a real-world energy forecasting task using supervised machine learning that demonstrates architecture blocks non-compliant and tampered artefacts while permitting valid transitions.

II. RELATED WORK

The application of blockchain for provenance in autonomous systems has evolved from simple timestamping to complex integration with Industrial IoT (IIoT) and Cyber-Physical Systems (CPS). We categorise the state-of-the-art into three distinct approaches as passive logging, active enforcement, and system-level integrity.

Blockchain-based provenance systems such as ProvChain [8] and SmartProvenance [9] established the foundational pattern of using distributed ledgers as tamper-evident notaries

for data lineage. This approach has since been adopted across multiple domains. In IoT environments, Kulothungan [11] proposed a ledger system to record AI decisions and sensor inputs, while Wang and Guan [12] developed cryptographic traceability schemes for secure data sharing. These systems provide strong forensic auditability by creating an immutable history of what occurred. However, they are fundamentally reactive in nature. They enable detection of violations post-facto but offer no mechanism to pre-emptively block the progression of non-compliant artefacts through the development lifecycle.

To move beyond passive logging, recent works have explored “Active Enforcement,” where smart contracts trigger automated actions based on ledger states. BlockFLA [13] and RBAC-SC [14] exemplify this approach by implementing role-based access control on-chain, thereby ensuring that only authorised users can submit model updates or access sensitive data. Zhuang et al. [15] apply this to healthcare, using contracts to enforce patient-centric access control policies dynamically. In the domain of smart cities, Villegas-Ch et al. [16] propose decentralized architectures where contracts automate governance decisions in urban IoT. Similarly, De Silva et al. [17] demonstrated this in robotics, using smart contracts to actively validate motion vectors against safety constraints before execution. While these systems successfully convert data into action, they typically focus on physical safety or access control rather than the *non-functional* properties of AI models (e.g., traceability, bias, drift, or fairness) required for high-risk applications.

A third line of research seeks to address this gap by providing mathematical guarantees about model behaviour through verifiable computation. Kang et al. [18] and Li et al. [19] have proposed cryptographic proof systems capable of verifying inference correctness without revealing model weights or input data. On the integrity front, Moradpoor et al. [20] propose a blockchain-based verification system for Federated Learning in ICS, ensuring that model updates are cryptographically signed and verified before aggregation. Bhuyar [21] further emphasises the role of mathematical cryptography in securing industrial automation transmissions. These approaches face two practical challenges. First, the computational overhead of generating and verifying proofs remains prohibitive for realistic AI lifecycles, particularly when models are frequently retrained. Second, the volume of data generated by high-frequency training and inference loops strains on-chain storage. Akrasi-Mensah et al. [22] address the latter by optimising storage in Blockchain-IIoT applications using deep reinforcement learning and propose hybrid on-chain/off-chain architectures. Although recent advances in Zero-Knowledge Machine Learning (ZK-ML) may eventually address the computational limitation, current implementations cannot yet support the throughput required for continuous integration pipelines.

Synthesising these three research directions reveals a persistent gap in AI lifecycle traceability. Passive logging provides auditability but cannot prevent non-compliant artefacts from progressing. Identity-based gating enforces access

control but does not evaluate the properties of submitted artefacts. Verifiable computation offers strong guarantees but remains computationally impractical for production environments. What remains missing is an approach that actively enforces constraints as a condition for lifecycle progression, while remaining efficient enough for practical deployment. Our approach addresses this gap by coupling cryptographic provenance with constraint enforcement. The Merkle-DAG captures the full AI lifecycle as a tamper-evident structure that serves as input to constraint evaluation. The smart contract ensures that only compliant transitions are anchored on-chain. By separating structural invariants enforced on-chain from dynamic constraints evaluated off-chain via OPA, the approach accommodates evolving requirements without compromising ledger integrity or incurring prohibitive computation costs.

III. METHODOLOGY

The proposed architecture has three components. The first is a Merkle-DAG that captures snapshots of the AI lifecycle state and stores artefacts off-chain. The second is a smart contract that enforces lifecycle transitions on-chain. The third is a trusted oracle implemented using Open Policy Agent (OPA), which evaluates constraints off-chain and provides signed attestations that the smart contract verifies. Provenance informs constraint evaluation, and enforcement guarantees the integrity of future provenance.

A. Merkle-DAG for AI Lifecycle Provenance

The Merkle-DAG captures snapshots of the AI lifecycle state at each stage. Unlike generic provenance structures, this Merkle-DAG represents the full AI lifecycle within a single tamper-evident structure. This includes technical artefacts such as datasets, features, models, metrics, and evaluation records. It also captures human interactions that occur beyond technical evaluation, including reviews, approvals, and compliance attestations. Each artefact is represented as a content-addressed node, with directed edges capturing derivation relationships where upstream artefacts act as parents and downstream artefacts as children. This structure aligns cryptographic hashing with the causal direction of the lifecycle, ensuring that each child node commits to the identities of all artefacts from which it was derived.

To reflect the non-linear structure of real AI workflows, the Merkle-DAG organises artefacts into three sub-graphs. The *Development Sub-Graph* contains raw data, cleaning scripts, engineered features, training configurations, and model weights. The *Evaluation Sub-Graph* contains inference outputs, metric logs, explanation artefacts, and summary reports. The *Compliance Sub-Graph* contains fairness assessments, reviewer identities, digital signatures, and constraint satisfaction records. These sub-graphs evolve in parallel and frequently intersect, producing branching and multi-parent relationships that are naturally supported by a DAG structure.

Each artefact A_i is serialised and hashed to obtain a unique digest. Lineage is established by embedding the hashes of parent artefacts into the hash of each successor node. For

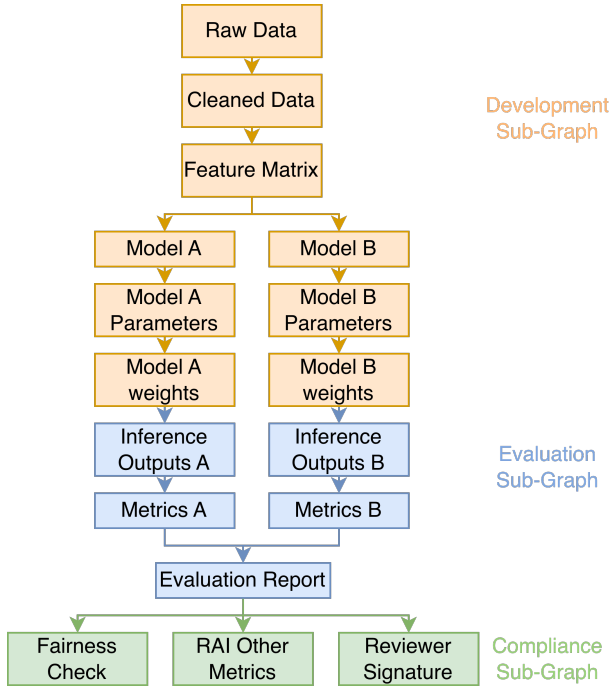


Fig. 1. Merkle-DAG structure for AI lifecycle provenance

example, if a model M is derived from dataset D and configuration C , its identity is computed as:

$$h(M) = \mathcal{H}(h(D) \parallel h(C) \parallel \text{metadata}) \quad (1)$$

where $\mathcal{H}$ is a cryptographic hash function and $\parallel$ denotes concatenation. Because each child embeds the digests of all its parents, any modification to an upstream artefact propagates forward through the graph, producing a different hash for all downstream nodes and rendering the resulting provenance unverifiable.

The storage architecture separates artefact content from cryptographic commitments. All artefacts, from raw data files and model weights to evaluation reports, are stored in the InterPlanetary File System (IPFS) as content-addressed JSON for Linked Data (JSON-LD) objects. IPFS provides distributed, immutable storage where each object is retrievable by its hash. The blockchain records only the 32-byte Merkle root R_t computed from the snapshot at lifecycle stage t , along with its linkage to the previous root ($R_{t-1} \rightarrow R_t$). This separation ensures that the complete provenance state can be reconstructed from IPFS at any time, while the on-chain root verifies the integrity of the off-chain data without incurring heavy storage costs on chain. Figure 1 illustrates the structure, showing how raw data, engineered features, model variants, and evaluation outputs form a branching Merkle-DAG.

B. Smart Contract for Lifecycle Enforcement

The smart contract ensures that lifecycle transitions satisfy defined constraints. It implements the lifecycle as a Finite State Machine (FSM) where transitions between states are gated by constraint evaluation. The current implementation

enforces two gates. The first gate, between the Development and Validated stages, enforces technical performance metrics such as RMSE thresholds. The second gate, between Validated and Approved, enforces fairness metrics and human oversight requirements. Additional gates can be added as requirements evolve. Figure 2 illustrates the interaction between the Merkle-DAG, OPA, and the smart contract.

A key architectural decision is the separation of policy enforcement into two layers. Hardcoding complex evaluation metrics directly into Solidity smart contracts is computationally expensive and operationally rigid, as even changing a threshold would require redeploying the contract. To address this, we decouple *structural invariants* from *dynamic constraints*. The smart contract enforces structural constraints that define the integrity of the system itself. These include verifying that the sender is authorised to request transitions, ensuring that the new Merkle root cryptographically links to the previous anchor, and validating the signature on the OPA attestation. These constraints are fixed and apply universally regardless of organisational context. Dynamic governance rules are expressed in Rego, a declarative policy language evaluated by OPA. These rules encode context-specific requirements such as performance thresholds, fairness criteria, and human oversight mandates. Because Rego policies are evaluated off-chain, organisations can update their governance standards in response to new regulations or evolving best practices without modifying the underlying smart contract infrastructure.

C. Trusted Oracle for constraint evaluation

Delegating constraint evaluation to an off-chain agent introduces the classic “Oracle Problem”, which is how can the blockchain trust that the OPA evaluation actually occurred? If the OPA engine runs on the data scientist’s local machine, the result could be fabricated. To address this, the OPA engine is deployed as a managed service within the consortium’s trust boundary and holds a distinct private key (K_{Oracle}^{Priv}). When OPA evaluates a policy as ‘Allow’, it signs the transition payload. The smart contract holds the corresponding public key (K_{Oracle}^{Pub}). The contract does not compute the metrics as it simply verifies the cryptographic signature. This ensures that while the request is initiated by the user, the validation is cryptographically guaranteed by the trusted oracle.

Figure 3 illustrates a representative OPA policy implemented in the Rego query language. This example demonstrates the Validated to Approved gate, which requires both a fairness assessment showing Disparate Impact within acceptable bounds and a signed attestation from a human reviewer confirming completion of a risk checklist.

This architecture achieves three objectives. First, by structuring provenance as a Merkle-DAG, it transforms mutable pipeline artefacts into tamper-evident commitments, enabling auditability and traceability. Second, by gating transitions through smart contracts, it replaces administrative trust with cryptographic proof, establishing accountability. Third, by delegating dynamic policy evaluation to OPA, it maintains

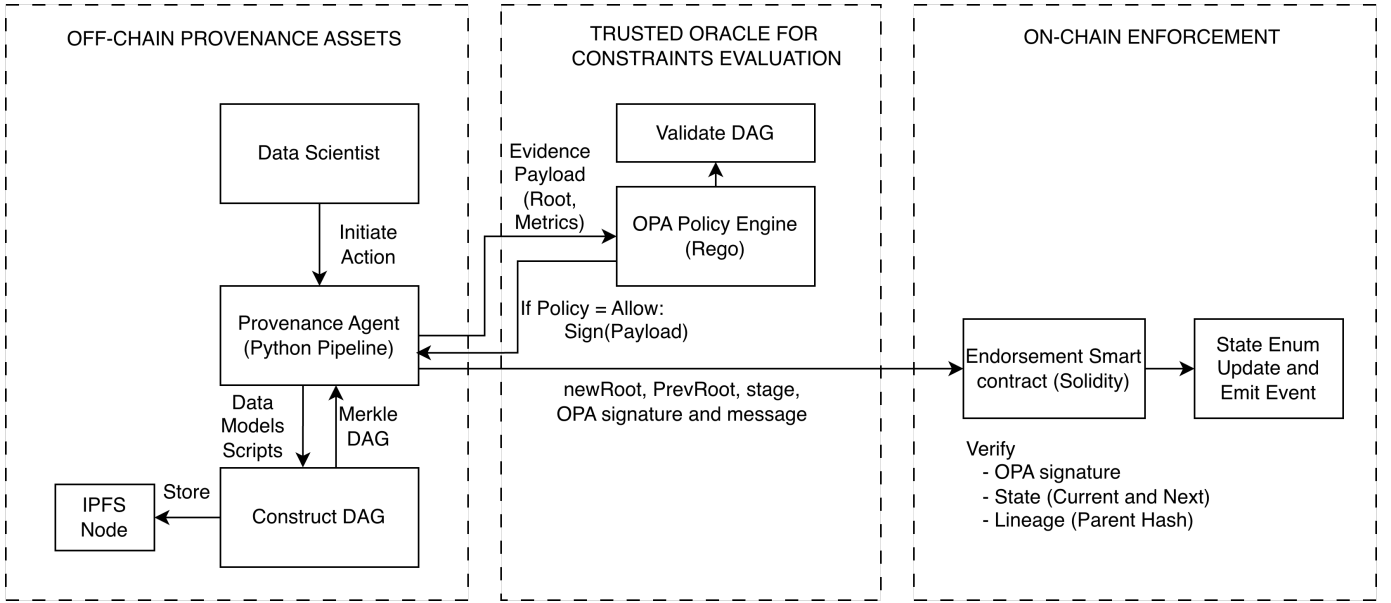


Fig. 2. Architecture showing Merkle-DAG provenance, OPA constraint evaluation, and smart contract enforcement

flexibility and scalability while preserving the integrity guarantees of the on-chain layer.

```
package lifecycle.validated_to_approved

default allow := false

allow if {
  input.model_id != ""
  # 1. Fairness: Disparate Impact [0.8, 1.25]
  input.fairness.disparate_impact >= 0.8
  input.fairness.disparate_impact <= 1.25

  # 2. Human Oversight
  input.review.reviewer_signature != ""
  input.review.risk_checklist.complete == true
}
```

Fig. 3. Rego policy for lifecycle transition constraints

IV. EXPERIMENTS

We implemented the proposed approach in an end-to-end energy forecasting pipeline deployed within the La Trobe Energy AI Platform (LEAP). LEAP is the core AI and data analytics technology stack in La Trobe University’s Net Zero Carbon Emissions Program, which aims to reduce the University’s carbon footprint to net zero by 2029. The platform operates within the University’s City of the Future living lab, a microgrid environment that generates building-level consumption, generation, and energy market data. The pipeline uses building-level consumption data comprising 5-minute interval readings across multiple buildings. The implementation integrates a Python provenance agent for Merkle-DAG construction, an OPA sidecar service for constraint evaluation, and a Solidity smart contract deployed on an Anvil local testnet for on-chain enforcement.

The contract stores the 32-byte Merkle root in a mapping structure, with metadata emitted via events to enable off-chain

indexers to reconstruct the audit trail without querying the active chain state. Lifecycle stages are encoded as Solidity enums to enforce a rigid FSM. The core transition function, `submitRoot`, applies three invariants before accepting any state change. First, the current on-chain stage must strictly precede the requested stage, preventing out-of-order transitions. Second, the submitted root must cryptographically link to the previous anchor, preserving the chain of custody. Third, the OPA attestation signature must be valid, ensuring that the trusted oracle has evaluated and approved the transition.

The experimental workflow begins in the Development stage. The Python pipeline ingests raw consumption data (D_{raw}) and trains an XGBoost regressor to forecast building-level energy demand. The provenance agent serialises the raw data, preprocessed data, feature engineering scripts, extracted features, and model weights to compute their SHA-256 hashes. These hashes are linked to form the DAG’s genesis branch, constituting the Development Sub-Graph.

A. Experiment 1: Metric Enforcement (*Dev* → *Validated*)

Upon completion of model training, the pipeline executes a test suite. To ensure the integrity of these results, the raw inference outputs and metric logs are hashed and incorporated into the Merkle-DAG as part of the Evaluation Sub-Graph. The pipeline then constructs an evidence package containing the candidate Merkle root and the computed metrics, and submits this package to the OPA engine.

The Rego policy enforces domain-specific acceptance criteria. For this campus-level energy model, the policy requires $RMSE < 50$ kW and $MAPE < 0.15$. If the reported metrics satisfy these constraints, OPA signs the evidence package and returns it to the pipeline. The pipeline then submits this signature and the new Merkle root to the smart contract. On-chain, the contract verifies the OPA signature, asserts that the

current state is `DEV`, and verifies that the provided Merkle root links correctly to the previous anchor. Only when all conditions are satisfied does the contract update the state to `VALIDATED` and anchor the new root.

B. Experiment 1 Results

We evaluated the approach against three scenarios designed to test whether cryptographic enforcement can block non-compliant artefacts while permitting valid transitions. These scenarios simulate three dominant failure modes in AI pipelines. The first is the baseline case where compliant artefacts should progress. The second tests whether non-compliant artefacts can bypass validation, a common failure in trust-based pipelines where practitioners can modify configuration to skip failing tests. The third tests whether tampered provenance can be submitted, a property many blockchain provenance papers claim but do not enforce during lifecycle transitions. The proposed architecture must cryptographically prevent all three failure modes.

Scenario 1 (Valid Performance Metrics): An XGBoost model was trained and evaluated, yielding an RMSE of 30 kW, within the threshold of 50 kW. OPA evaluated the evidence package and returned an `allow` decision with a valid signature. The smart contract verified the attestation and anchored the new Merkle root, updating the state to `VALIDATED`. This confirms that compliant artefacts progress as expected.

Scenario 2 (Invalid Performance Metrics): We simulated model degradation by submitting metrics that failed requirements, with an RMSE of 300 kW. OPA evaluated the constraint as `deny` and did not issue an attestation. The pipeline halted without submitting any transaction to the blockchain. This demonstrates pre-emptive filtering: non-compliant artefacts are blocked at the OPA layer, preventing state pollution on-chain and eliminating gas costs for invalid transitions.

Scenario 3 (Tampered Provenance): We attempted to submit a transition using a manually modified Merkle root that did not correspond to the actual provenance state. The smart contract detected the mismatch between the submitted root and the expected derivation from the previous anchor. The request was rejected before any on-chain state change occurred. This confirms that cryptographic linkage between roots provides tamper detection.

Together, these three scenarios demonstrate that the proposed system enforces metric-based constraints pre-emptively, preventing non-compliant models from entering downstream stages.

C. Experiment 2: Human Oversight (Validated $\rightarrow$ Approved)

The final transition to `APPROVED` demonstrates how the approach can enforce requirements beyond automated performance metrics. This gate combines automated fairness assessment with human-in-the-loop oversight, illustrating that additional constraints can be integrated through OPA policies without modifying the smart contract. The OPA policy for this transition validates two conditions. First, a fairness report must demonstrate that Disparate Impact remains within acceptable

bounds (between 0.8 and 1.25). This fairness evaluation integrates logic from our Agentic Debiasing framework [23], showing how external responsible AI tooling can be incorporated into the governance pipeline. Second, a verified human reviewer must have digitally signed a review checklist confirming manual review of the model. The reviewer's signature is validated off-chain by OPA. Only when both conditions are met does OPA issue an approval attestation.

If the fairness report is missing from the Merkle-DAG lineage, or if the bias metrics exceed the threshold, or if the human review signature is absent, OPA denies the attestation immediately, providing pre-emptive filtering capabilities. No invalid transaction reaches the blockchain, and the pipeline cannot progress until requirements are satisfied.

D. Experiment 2 Results

Scenario 4 (Valid Human Oversight): We tested the `VALIDATED` to `APPROVED` transition with a complete evidence package containing a fairness report showing Disparate Impact of 1.1 and a valid human reviewer signature. OPA verified both conditions and issued an approval attestation. The smart contract verified the signature and finalised the lifecycle state to `APPROVED`. This demonstrates that the approach can enforce requirements that extend beyond automated metrics to include human oversight.

TABLE I
CONSTRAINT ENFORCEMENT OUTCOMES

Scenario	OPA Decision	Chain Result	Outcome
1. Valid Metrics	Allow = True	State Updated	PASS
2. Invalid Metrics	Allow = False	<i>No Tx Submitted</i>	PASS
3. Tampered Root	Allow = False	<i>No Tx Submitted</i>	PASS
4. Valid Approval	Allow = True	State Updated	PASS

These results validate the core claim that coupling provenance with constraint enforcement enables active traceability. In a conventional MLOps pipeline, Scenarios 2 and 3 would not necessarily be blocked. A practitioner could bypass failing tests by modifying pipeline configuration, or submit altered artefacts without detection if lineage tracking relies on mutable logs. In our approach, such actions are cryptographically impossible. The Merkle-DAG structure ensures that any modification invalidates downstream hashes, and the smart contract refuses transitions without valid OPA attestations.

V. DISCUSSION AND CONCLUSION

This paper addresses the challenge of AI lifecycle traceability in high-risk applications by proposing a novel method that captures AI lifecycle state in a Merkle-DAG structure that supports provenance, constraint evaluation, and on-chain enforcement. The Merkle-DAG records the complete lineage of artefacts from datasets and features to models, metrics, and human approvals. An OPA oracle evaluates constraints to state transition off-chain and produces signed attestations. A smart contract verifies these attestations and anchors the Merkle root on-chain, ensuring that only compliant artefacts progress through the lifecycle. The experimental results validate this

approach. While scenario 2 demonstrated that non-compliant artefacts are blocked before reaching the blockchain, Scenario 3 demonstrated that tampered provenance is rejected before any state change occurs. These results show that the architecture prevents three dominant failure modes in AI pipelines for invalid artefacts progressing, tampered artefacts progressing, and undocumented modifications escaping detection.

The Merkle-DAG and smart contract components achieve traceability properties that traditional MLOps tooling supports only through administrative controls. Auditability is achieved through the tamper-evident Merkle-DAG structure, which commits to the complete lineage of every artefact. Traceability follows from the cryptographic linkage between Merkle roots, enabling reconstruction of the full derivation path from any deployed model back to its source data. Accountability is enforced through constraint-gated transitions, ensuring that lifecycle progression requires explicit satisfaction of defined requirements. Transparency emerges from the on-chain record of anchored roots and emitted events. Beyond these properties, the approach supports integration of additional constraints through OPA policies, as demonstrated by the fairness and human oversight gate in Experiment 2. Organisations can incorporate bias metrics, review requirements, or other checks by defining appropriate Rego policies without modifying the smart contract.

The proposed Merkle-DAG captures the dependency structure and state transitions of the AI lifecycle (*process provenance*) rather than serving as a data storage mechanism. The approach assumes external infrastructure for storing datasets and artefacts referenced by the graph. This separation keeps provenance logic lightweight, while the design of data infrastructure that preserves *data provenance* across transformations remains a complementary challenge. Additionally, The current implementation operates as a proof-of-concept on a local testnet, and further work is required to evaluate performance in production blockchain environments. Also, the architecture currently relies on a trusted OPA oracle for off-chain constraint evaluation, reflecting a common design trade-off in hybrid on-chain/off-chain systems. Future work will address this by integrating Zero-Knowledge Machine Learning (ZK-ML) techniques, enabling cryptographic proof that constraints were correctly evaluated and eliminating the need to trust the oracle.

ACKNOWLEDGMENT

This work was supported by the Department of Climate Change, Energy, the Environment and Water of the Australian Federal Government, as part of the International Clean Innovation Researcher Networks (ICIRN) program, grant number ICIRN000077.

REFERENCES

- [1] D. De Silva, O. Kaynak, M. El-Ayoubi, N. Mills, D. Alahakoon, and M. Manic, "Opportunities and challenges of generative artificial intelligence: Research, education, industry engagement, and social impact," *IEEE Industrial Electronics Magazine*, 2024.
- [2] D. Rolnick, P. L. Donti, L. H. Kaack, *et al.*, "Tackling climate change with machine learning," *ACM Computing Surveys*, vol. 55, no. 2, pp. 1–96, 2022.
- [3] M. Anisetti, C. A. Ardagna, E. Damiani, and P. G. Panero, "A methodology for non-functional property evaluation of machine learning models," in *Proceedings of the 12th International Conference on Management of Digital EcoSystems*, MEDES '20, (New York, NY, USA), p. 38–45, Association for Computing Machinery, 2020.
- [4] M. Herschel, R. Diestelkämper, and H. Ben Lahmar, "A survey on provenance: What for? what form? what from?," vol. 26, pp. 881–906, 12 2017.
- [5] M. Schlegel and K.-U. Sattler, "Capturing end-to-end provenance for machine learning pipelines," *Information Systems*, vol. 132, p. 102495, 2025.
- [6] M. Ahmed, A. Dar, M. Helfert, A. Khan, and J. Kim, "Data provenance in healthcare: Approaches, challenges, and future directions," *Sensors*, vol. 23, no. 14, p. 6495, 2023.
- [7] Z. Zheng, S. Xie, H.-N. Dai, W. Chen, X. Chen, J. Weng, and M. Imran, "An overview on smart contracts: Challenges, advances and platforms," *Future Generation Computer Systems*, vol. 105, pp. 475–491, 2020.
- [8] X. Liang, S. Shetty, D. Tosh, C. Kamhoua, K. Kwiat, and L. Njilla, "Provchain: A blockchain-based data provenance architecture in cloud environment with enhanced privacy and availability," in *Proceedings of the 17th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid)*, pp. 468–477, IEEE, 2017.
- [9] A. Ramachandran and M. Kantarcioglu, "Smartprovenance: A distributed provenance framework for data science pipelines," in *Proceedings of the 8th ACM Conference on Data and Application Security and Privacy (CODASPY)*, pp. 354–360, ACM, 2018.
- [10] M. Kandpal, V. Goswami, R. Priyadarshini, and R. Barik, "Towards data storage, scalability, and availability in blockchain systems: A bibliometric analysis," *Data*, vol. 8, no. 10, p. 148, 2023.
- [11] V. Kulothungan, "Using blockchain ledgers to record ai decisions in iot," *IoT*, vol. 6, no. 3, 2025.
- [12] Z. Wang and S. Guan, "A blockchain-based traceable and secure data-sharing scheme," *PeerJ Computer Science*, 2023.
- [13] H. Desai, M. S. Ozdayi, and M. Kantarcioglu, "Blockfla: Accountable federated learning via hybrid blockchain architecture," in *Proceedings of the 2021 IEEE International Conference on Data Mining (ICDM)*, pp. 1054–1059, IEEE, 2021.
- [14] J. P. Cruz, Y. Kaji, and N. Yanai, "Rbac-sc: Role-based access control using smart contract," *IEEE Access*, vol. 6, pp. 12240–12251, 2018.
- [15] Y. Zhuang, L. R. Sheets, Y.-W. Chen, Z. Shae, J. J. P. Tsai, and C.-R. Shyu, "A patient-centric health information exchange framework using blockchain technology," *IEEE Journal of Biomedical and Health Informatics*, vol. 24, pp. 2169–2176, 2020.
- [16] W. Villegas-Ch, R. Gutierrez, J. Govea, and P. Palacios, "Design and evaluation of a decentralized urban governance system with embedded ai and blockchain-enabled iot," *Frontiers in Sustainable Cities*, vol. 7, 2025.
- [17] D. De Silva, S. Withanage, V. Sumanasena, L. Gunasekara, H. Moraliyage, N. Mills, and M. Manic, "Robotic motion intelligence using vector symbolic architectures and blockchain-based smart contracts," *Robotics*, vol. 14, no. 4, 2025.
- [18] D. Kang, T. Hashimoto, I. Stoica, and X. Yi, "Scaling up trustless dnn inference with zero-knowledge proofs," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 375–392, ACM, 2021.
- [19] X. Li, J. He, P. Vijayakumar, X. Zhang, and V. Chang, "A verifiable privacy-preserving machine learning prediction scheme for edge-enhanced hcps," *IEEE Transactions on Industrial Informatics*, vol. 18, pp. 5494–5503, 2022.
- [20] N. Moradpoor, M. Barati, A. Robles-Durazno, E. Abah, and J. McWhinnie, "Neutralizing adversarial machine learning in industrial control systems using blockchain," pp. 437–451, 2023.
- [21] D. Bhuyar, "Blockchain-enabled secure data transmission for industrial automation: a mathematical cryptography perspective," *International Journal of Applied Mathematics*, vol. 38, no. 8s, pp. 893–906, 2025.
- [22] N. Akrafi-Mensah *et al.*, "Adaptive storage optimization scheme for blockchain-iiot applications using deep reinforcement learning," *IEEE Access*, vol. 11, pp. 1372–1385, 2023.
- [23] J. N. Gunaweera, L. Gunasekara, and H. D. Rajapakse, "A generative agentic ai framework for analysis and debiasing of datasets and machine learning models," in *IECON 2025 – 51st Annual Conference of the IEEE Industrial Electronics Society*, pp. 1–6, 2025.